EEL 5764: Computer Architecture

Lectures 10–11: RISC-V Single-Cycle Datapath & Control

Sandip Ray
Warren B. Nelms Endowed Professor
Department of Electrical & Computer Engineering, University of Florida | Office: MALA 4131
Required Reading: H&P Appendix C, P&H Chapter 4

Bridging ISA to Hardware: The Microarchitect's Task

THE HARDWARE-SOFTWARE INTERFACE BOUNDARY

THE LOGICAL CONTRACT

- **ISA State Definition:** The hardware must maintain and update the architectural state: the PC, Register File (x1-x31), and Memory.
- **Functional Expectations:** Any executed binary must yield the exact register and memory state transformations specified in the instruction manual.
- **The Implementation Horizon:** The microarchitecture has complete freedom in layout (pipelining, caching, branch predictors) as long as function holds.

THE SILICON REALIZATION

- **Building block components:** We will assemble physical structures—SRAM register files, ALU adders, multiplexers, and state registers.
- **Timing and clock limits:** We must ensure signals propagate through combinational gates and settle before the next active clock edge.
- **Control logic routing:** We will design the 'brain' of the CPU, decoding opcodes into static control wires that open and close datapath multiplexers.

High-Level Datapath Blueprint: The Execution Cycle

1. IF

Instruction Fetch

Read 32-bit instruction from Inst Memory at PC address.

2. ID

Instruction Decode

Parse opcode/funct bits & read operands from Reg File in parallel.

3. EX

Execute / ALU

Perform arithmetic, logic, or address displacement via ALU.

4. MEM

Memory Access

Read or write Data Memory if executing lw/sw; else bypass.

5. WB

Write-Back

Write final result (ALU or memory value) back to rd in Reg File.

Single-Cycle Principle: In a single-cycle implementation, all 5 stages execute continuously within a single clock period. The active clock edge marks the boundary between instructions.

Storage Elements: State Elements vs. Combinational Logic

COMBINATIONAL LOGIC

- **No internal memory state:** Output is strictly a function of current inputs (e.g., ALU, adders, multiplexers, decoders).
- **Propagation Delay limits:** Logic gates have physical resistance and capacitance that delay signal transitions from input to output.
- **Continuous transformation:** Combinational blocks process data continuously; they do not wait for clock ticks to compute.

STATE STORAGE ELEMENTS

- **Clock-Edge Triggered:** Storage blocks (like PC or Register File) only update their values on the active edge of the clock.
- **Write-Enable Signals:** Dedicated control lines (RegWrite, MemWrite) dictate whether a state element accepts a new value at the clock edge.
- **SRAM Register Files:** Multi-ported memory structures that allow reading two registers and writing one register in parallel.

Instruction Fetch (IF) Logic

THE INSTRUCTION FETCH PIPELINE LOOP

PROGRAM COUNTER (PC)

- **32-bit State Register:** Holds the memory address of the instruction currently being executed.
- **Clock Edge Triggered:** Updates value on rising clock edge.
- **Sequential Pointer:** Serves as the starting anchor for all datapath operations.

INSTRUCTION MEMORY

- **Read-Only SRAM Block:** Contains the compiled binary executable.
- **Displacement Lookup:** Accepts 32-bit PC address as input.
- **32-Bit Output:** Instantly outputs instruction word to Decode stage.

PC INCREMENT ADDER

- **Dedicated Adder:** Continuously calculates PC + 4.
- **Word Boundaries:** Increments address by 4 bytes (32 bits).
- **Default Next PC:** Feeds sequential execution target back to PC multiplexer.

Arithmetic/Logical (R-Type) Datapath

REGISTER-REGISTER COMPUTATIONAL EXECUTION (e.g. add, sub, and, or)

REGISTER FILE PORTS

- **Dual Read Ports:** The Register File outputs two 32-bit values from source registers rs1 (bits 19:15) and rs2 (bits 24:20) in parallel.
- **Single Write Port:** Accepts destination register rd (bits 11:7) address and 32-bit Write Data from ALU.
- **RegWrite Control:** High write-enable signal enables writing the ALU calculation to rd at the rising clock edge.

THE ALU INTEGRATION

- **Arithmetic Logic Unit (ALU):** Takes two 32-bit register inputs and performs arithmetic/logical operations.
- **ALU Control Lines:** 4-bit control signal specifying operation (ADD, SUB, AND, OR, SLT, XOR).
- **Zero Flag Output:** Dedicated output bit set to 1 if the ALU result is zero (vital for branch comparisons).

Immediate Arithmetic (I-Type) Datapath

SIGN-EXTENDER BLOCK

- **12-bit Immediate Extraction:** Pulls bits [31:20] from I-type instruction.
- **Sign Extension:** Replicates bit 31 across upper 20 bits.
- **32-bit Output:** Converts 12-bit signed constant (-2048 to +2047) to 32-bit integer.

ALUSRC MULTIPLEXER

- **2-to-1 Mux Selection:** Positioned at second ALU input port.
- **ALUSrc = 0:** Selects Register Read Data 2 (for R-type instructions).
- **ALUSrc = 1:** Selects 32-bit Sign-Extended Immediate (for I-type instructions).

ALU & WRITE-BACK

- **Immediate Calculation:** ALU adds/operates on rs1 data and immediate.
- **Write-Back Path:** ALU result routes directly to Register File Write Data port.
- **RegWrite High:** Writes calculated result into destination register rd.

Memory Load Datapath (lw)

DATA MEMORY FETCH EXECUTION (e.g. lw rd, offset(rs1))

ADDRESS COMPUTATION

- **Displacement Arithmetic:** Loads calculate effective memory address as Base Register (rs1) + Sign-Extended 12-bit Offset.
- **ALUSrc = 1:** Routes sign-extended offset into second ALU input.
- **ALU Add Operation:** ALU performs addition to compute absolute Data Memory byte address.

DATA MEMORY ACCESS & WRITE-BACK

- **MemRead Control:** High read-enable signal triggers Data Memory to output 32-bit word from calculated address.
- **MemtoReg Multiplexer:** Selects Data Memory output (MemtoReg = 1) instead of ALU result (MemtoReg = 0).
- **RegWrite High:** Writes retrieved memory data into destination register rd at clock edge.

Memory Store Datapath (sw)

DATA MEMORY WRITE EXECUTION (e.g. sw rs2, offset(rs1))

ADDRESS GENERATION

- **Split-Immediate Extraction:** The 12-bit offset is extracted from instruction bits [31:25] and [11:7] and sign-extended.
- **Base Address Addition:** ALU adds Base Register (rs1) data and sign-extended offset to compute effective memory write address.
- **MemWrite Control:** High write-enable signal enables Data Memory to accept incoming data at rising clock edge.

WRITING TO MEMORY

- **Store Data Source:** Data to be written comes directly from Register Read Data 2 (rs2).
- **Direct Memory Connection:** Read Data 2 bypasses ALU and connects directly to Data Memory Write Data input.
- **RegWrite Low:** RegWrite is set to 0 during stores to prevent corrupting Register File state.

The Memory-Write Path Challenge

HARDWARE-COMPILER CO-DESIGN: THE SPLIT-IMMEDIATE CHOICE

S-TYPE PHYSICAL ALIGNMENT

- **Register Bit Integrity:** In S-type, source register specifiers stay locked at rs1 [19:15] and rs2 [24:20].
- **Split Immediate Offset:** The 12-bit store offset is split across bits [31:25] (imm[11:5]) and [11:7] (imm[4:0]).
- **Compiler Responsibility:** The compiler toolchain splits the immediate during instruction encoding.

PORT AND WIRE COMPARISON

- **Mux-Free Register Port:** By preserving rs2 at [24:20], Register Read Port 2 outputs store data directly to Data Memory.
- **Bypassing Multiplexers:** Eliminates an input multiplexer on the Register File output, saving physical silicon area.
- **Reduced Decode Latency:** Allows Register File access to occur instantly in parallel with opcode decoding.

Branch Datapath (beq/bne)

PC-RELATIVE TARGET ADDER

- **B-Type Immediate:** Scrambled 12-bit offset sign-extended and left-shifted by 1 bit (halfword alignment).
- **Dedicated Branch Adder:** Computes Target = PC + (Sign-Extended Immediate << 1).
- **Parallel Add:** Computes target address in parallel with ALU comparison.

ALU ZERO COMPARATOR

- **ALU Subtraction:** ALU computes rs1 - rs2 to test equality.
- **Zero Flag Output:** Sets Zero = 1 if rs1 == rs2 (result is 0).
- **AND Gate Logic:** Branch Decision = Branch Control Line AND Zero Flag.

PCSEL MULTIPLEXER

- **PCSel Control:** Selects next PC source for instruction fetch.
- **PCSel = 0 (Default):** Selects sequential PC+4 address.
- **PCSel = 1 (Branch Taken):** Selects Branch Target Address from Branch Adder.

Unconditional Jumps (jal/jalr)

PC-RELATIVE JUMP (JAL)

- **Direct PC Offset:** jal computes its target address as PC + Sign-Extended 20-bit Immediate (J-type).
- **Wide Jump Range:** PC-relative jumps span +/- 1 Megabyte address space.
- **Target Multiplexer:** Target address feeds directly into PC selection multiplexer.

INDIRECT JUMPS (JALR)

- **Register-Indirect Target:** jalr computes target as Base Register (rs1) + Sign-Extended 12-bit Offset (I-type).
- **32-Bit Target Span:** Enables jumping to any absolute 32-bit address in memory.
- **Function Pointers:** Primary mechanism for dynamic function pointers and subroutine returns (jalr x0, 0(ra)).

Link Register Write-Back: Both jal and jalr write return address (PC+4) to destination register rd (typically ra/x1).

Unifying the Datapath: The Unified Single-Cycle Diagram

THE UNIFIED SINGLE-CYCLE ARCHITECTURE GRID

- **Connecting the Loops:** We merge the instruction fetch loop, register file, ALU, data memory, sign extender, and multiplexers into a single unified execution grid.
- **The Dynamic Routing Principle:** All physical wires and functional blocks are active continuously on every clock cycle. Multiplexers select which wire pathways carry valid data for the specific instruction being executed.
- **Clock-Edge Synchronization:** State elements (PC, Register File, Data Memory) hold state stable throughout the cycle, allowing combinational signals to propagate and settle before the next clock tick.

Unified Interconnect

100% Shared Silicon

4 Routing Muxes

Dynamic Control Lines

1 Clock Cycle

Single-Period Execution

Datapath Multiplexers: Routing Control

PCSEL (PC SELECTOR)

- **PCSel = 0:** Selects sequential PC + 4.
- **PCSel = 1:** Selects Branch/Jump target address.
- Controls next instruction fetch address.

IMMSEL (IMMEDIATE DECODER)

- Controls Sign-Extender decoding mode.
- Extracts I, S, B, U, or J-type immediate bits.
- Sign-extends offset to full 32-bit width.

ALUSRC (ALU SECOND OPERAND)

- **ALUSrc = 0:** Selects Register Read Data 2 (R-type).
- **ALUSrc = 1:** Selects 32-bit Sign-Extended Immediate.
- Feeds second operand input to main ALU.

WDSEL / MEMTOREG (WRITE-BACK)

- **MemtoReg = 0:** Selects ALU Result (arithmetic).
- **MemtoReg = 1:** Selects Data Memory output (loads).
- **MemtoReg = 2:** Selects PC + 4 (jump link address).

Designing the Control Unit: The Brain

TWO-LEVEL CONTROL DECODER ARCHITECTURE

THE MAIN DECODER

- **Opcode-driven logic:** The main decoder inspects instruction opcode bits [6:0] to generate static control signals.
- **Control Line Outputs:** Generates RegWrite, ALUSrc, MemRead, MemWrite, MemtoReg, Branch, Jump, and 2-bit ALUOp.
- **Combinational Truth Table:** Implemented as a simple, low-latency ROM or PLA lookup table in silicon.

ALU DECODER CO-DESIGN

- **Two-Level Decoding:** To keep the main decoder small and fast, ALU control is split into a secondary decoder.
- **ALUOp Signals:** Main decoder outputs 2-bit ALUOp (00 = Add for loads/stores, 01 = Sub for branches, 10 = R-type).
- **4-Bit ALU Control:** Secondary ALU decoder inspects ALUOp + funct3 + funct7 bits to output exact 4-bit ALU action.

Control Line Logic Truth Table

The physical mapping of high-level instruction opcodes to static datapath control line voltages:

Instruction Class	RegWrite	ALUSrc	MemRead	MemWrite	MemtoReg	Branch	ALUOp
R-Type (e.g., add, sub)	1	0	0	0	0	0	10 (R-Type)
I-Type (e.g., addi)	1	1	0	0	0	0	10 (R-Type)
Load (e.g., lw)	1	1	1	0	1	0	00 (Add)
Store (e.g., sw)	0	1	0	1	X	0	00 (Add)
Branch (e.g., beq)	0	0	0	0	X	1	01 (Sub)
Jump (e.g., jal)	1	X	0	0	2 (PC+4)	0	XX

Truth Table Insights: Note how RegWrite is 1 only for instructions that update registers (R-type, I-type, loads, jumps). MemtoReg is 1 only for loads. Don't-care conditions (X) allow logic synthesis tools to minimize gate counts.

ALU Control Logic: Multi-Level Decoding

Two-level decoding translating opcode actions and instruction funct fields to ALU control signals:

Instruction	ALUOp	Funct3	Funct7 (Bit 30)	Operation	ALU Control	Action
lw / sw	00	XXX	X	Addition	0010	Add base + offset
beq / bne	01	XXX	X	Subtraction	0110	Subtract for Zero flag
add	10	000	0	Addition	0010	Add rs1 + rs2
sub	10	000	1	Subtraction	0110	Subtract rs1 - rs2
and	10	111	0	Bitwise AND	0000	AND rs1 & rs2
or	10	110	0	Bitwise OR	0001	OR rs1 | rs2
slt	10	010	0	Set Less Than	0111	Compare rs1 < rs2

Multi-Level Decoder Efficiency: Bypasses inspecting funct fields for loads, stores, and branches, keeping the primary opcode decoder small and fast.

Single-Cycle Execution Walkthrough: R-Type (add)

STEP-BY-STEP DATAPATH EXECUTION FLOW

- **1. Instruction Fetch (IF):** PC outputs address to Instruction Memory, fetching 32-bit add instruction. PC+4 is calculated.
- **2. Decode & Reg Read (ID):** Main Decoder reads opcode 0110011, sets RegWrite=1, ALUSrc=0, MemRead=0, MemWrite=0, MemtoReg=0. Read Ports 1 and 2 output rs1 and rs2 values.
- **3. ALU Execution (EX):** Secondary ALU Decoder inspects funct3 (000) and funct7 (0000000), outputs ALU control 0010 (ADD). ALU computes rs1 + rs2.
- **4. Memory Bypass (MEM):** Data Memory is idle (MemRead=0, MemWrite=0). ALU result passes directly through MemtoReg multiplexer.
- **5. Write-Back (WB):** On the rising clock edge, ALU result is written into destination register rd. PC updates to PC+4.

ACTIVE CONTROL

RegWrite

1

ALUSrc

0 (Reg)

MemRead

0

MemWrite

0

MemtoReg

0 (ALU)

Single-Cycle Execution Walkthrough: Load (lw)

STEP-BY-STEP DATAPATH EXECUTION FLOW

- **1. Instruction Fetch (IF):** PC outputs address to Instruction Memory, fetching 32-bit lw instruction. PC+4 is calculated.
- **2. Decode & Reg Read (ID):** Main Decoder reads opcode 0000011, sets RegWrite=1, ALUSrc=1, MemRead=1, MemWrite=0, MemtoReg=1. Read Port 1 outputs rs1 base address. Sign-Extender expands 12-bit offset.
- **3. Address Calculation (EX):** ALUSrc mux selects 32-bit sign-extended offset. ALU Control 0010 (ADD) adds base address + offset.
- **4. Data Memory Fetch (MEM):** ALU result drives Data Memory address lines. MemRead=1 triggers memory to output 32-bit word from calculated address.
- **5. Write-Back (WB):** MemtoReg mux selects Data Memory output. On the rising clock edge, memory data is written into rd. PC updates to PC+4.

ACTIVE CONTROL

RegWrite

1

ALUSrc

1 (Imm)

MemRead

1

MemWrite

0

MemtoReg

1 (Mem)

Single-Cycle Execution Walkthrough: Branch (beq)

STEP-BY-STEP DATAPATH EXECUTION FLOW

- **1. Instruction Fetch (IF):** PC outputs address to Instruction Memory, fetching 32-bit beq instruction. PC+4 is calculated in parallel.
- **2. Decode & Reg Read (ID):** Main Decoder reads opcode 1100011, sets RegWrite=0, ALUSrc=0, MemRead=0, MemWrite=0, Branch=1. Read Ports 1 and 2 output rs1 and rs2 values. Sign-Extender expands B-type branch offset.
- **3. Comparison & Target Add (EX):** ALU Control 0110 (SUB) computes rs1 - rs2. Zero flag is set to 1 if rs1 == rs2. Dedicated Branch Adder computes Target = PC + (Imm << 1).
- **4. Branch Decision (MEM):** AND gate computes PCSel = Branch (1) AND Zero (1) = 1. PCSel mux selects Branch Target Address instead of PC+4.
- **5. State Update (WB):** RegWrite=0 and MemWrite=0 prevent state updates. On rising clock edge, PC updates to Branch Target Address.

ACTIVE CONTROL

RegWrite

0

ALUSrc

0 (Reg)

MemRead

0

MemWrite

0

Branch

1

Timing Analysis: Defining the Clock Period

T_clock >= T_PC_setup + T_InstMem + T_RegRead + T_ALU + T_DataMem + T_Mux + T_RegSetup

CRITICAL PATH DELAY SEQUENCE

- **Worst-Case Delay Path:** In a single-cycle design, the clock period T_clock is strictly constrained by the longest instruction path in the ISA: the Load (lw) instruction.
- **Sequential Accumulation:** The clock period must equal the sum of all propagation delays through Instruction Memory, Register File read, ALU address computation, Data Memory read, Write-back Mux, and Register File setup time.
- **Setup & Hold Constraints:** All combinational signals must settle at destination register inputs before the active clock edge.

TIMING BUDGET EXAMPLE

- **Assigned gate delays:** PC-setup = 10ps, InstMem = 200ps, RegRead = 100ps, ALU = 150ps, DataMem = 200ps, Mux = 30ps, RegSetup = 10ps.
- **Load Critical Path Delay:** T_lw = 10 + 200 + 100 + 150 + 200 + 30 + 10 = 700 picoseconds (ps).
- **Maximum Operating Frequency:** f_max = 1 / 700ps = 1.43 Gigahertz (GHz).

The Single-Cycle Bottleneck

WORST-CASE CLOCK PENALTY

- **Load Dictates Clock:** The clock cycle must be stretched to accommodate the load instruction (700ps).
- **Inflexible Period:** Every single instruction—no matter how simple—takes the full 700ps.
- **Violates Common Case Rule:** Violates the core principle of making the common case fast.

IDLE HARDWARE WASTED TIME

- **ADD Execution Time:** R-type instructions complete in 490ps (no Data Memory access).
- **Wasted Idle Window:** R-type instructions sit completely idle for 210ps (30% of every cycle) waiting for clock edge.
- **Branch Wasted Time:** Branches finish in 460ps, wasting 240ps (34% of cycle) per instruction.

REVOLUTIONARY FIX: PIPELINING

- **Slicing the Datapath:** Insert state registers between the 5 execution stages (IF, ID, EX, MEM, WB).
- **Stage-Based Clocking:** Clock period shrinks to match the longest single stage (~200ps) rather than total sum.
- **5x Throughput Boost:** Enables 5 instructions to execute simultaneously in overlapping stages!

Challenge: Designing a Floating-Point Store

ACTIVE ARCHITECTURE CHALLENGE: SPECIALIZED FP STORE INSTRUCTION

THE CUSTOM INSTRUCTION PROPOSAL

- **Specialized Store:** Your compiler team proposes adding a custom floating-point store instruction (fsw) that writes directly from FP Register File to Data Memory with auto-increment address calculation.
- **Hardware Logic Cost:** Adding this specialized indexing addressing logic adds 40ps of extra propagation delay to the main ALU.
- **Workload Impact:** The new instruction reduces overall program instruction count (IC) by 10%.

QUANTITATIVE EVALUATION

- **Original system parameters:** Operating clock period T_old = 700ps.
- **New system clock period:** New Load critical path T_new = 700ps + 40ps = 740ps.
- **Execution Time Ratio:** CPU_Time_new / CPU_Time_old = (0.90 * IC * 1 * 740ps) / (1.00 * IC * 1 * 700ps) = 666 / 700 = 0.951.

Verdict: System is 5.1% faster! The 10% reduction in IC outweighs the 5.7% clock slowdown.